Part 1: Securing GraphQL Vulnerabilities

GraphQL is very flexible because clients can decide what data they want and how deeply they want to access connected information. While this flexibility is useful, it also creates situations where attackers can misuse the system. To secure GraphQL, the server must stop trusting every request automatically and instead apply strict rules before processing queries.

---

1. Securing against Introspection Abuse

The Main Idea

Introspection is a built-in GraphQL feature that allows users to ask the server about its internal structure. It acts like a guidebook that explains all available queries, fields, relationships, and data types inside the API.

This feature is useful for developers because it helps them understand how the API works while building applications. However, attackers can also use the same feature to study the backend structure.

To secure against this problem, the server is configured to stop revealing its internal structure publicly.

How the Protection Works

The GraphQL server contains settings that control whether introspection is allowed or not. By changing this setting, the server can refuse introspection requests coming from outside users.

When the setting is disabled, normal application requests still work correctly. Users can still perform valid operations such as fetching profiles or updating data. However, if someone tries to ask for schema details or hidden structure information, the server immediately blocks the request.

Instead of returning the API structure, the server responds with an error or behaves as if the schema information does not exist.

This protection removes the attacker’s ability to explore the backend architecture freely.

Environment-Based Configuration

In most real systems, developers still need introspection during development and testing. Because of this, the setting is usually controlled separately for different environments.

On developers' local machines, introspection remains enabled so they can work easily while building the application.

Once the project is deployed to the public internet, the production server disables introspection automatically. This prevents outsiders from accessing internal API details while still allowing developers to work efficiently during development.

The idea is simple: trusted developers can view the structure, but public users cannot.

---

2. Securing against Depth/Circular DoS

The Main Idea

GraphQL allows clients to request connected data inside a single query. Because objects can reference each other repeatedly, attackers can create extremely deep requests that force the server into excessive processing.

To stop this, the server introduces limits on how deeply a query is allowed to go.

Instead of allowing unlimited nesting, the server checks the query structure before executing it.

How the Protection Works

Before the query reaches the database, the server performs an inspection process called validation.

During this process, the server analyzes the incoming query text and measures how deeply the nested fields go. It counts how many layers exist inside the request structure.

For example, the server may allow only five levels of nesting. If a query contains more than five levels, the server rejects it immediately.

The request never reaches the database and no heavy processing begins.

Why This Stops the Attack

Without limits, the server may repeatedly follow relationships again and again, creating thousands of internal operations from one request.

By introducing a depth limit, the server prevents extremely large recursive queries from running.

The server effectively says:

“You can request connected data, but only up to a safe level.”

This protection prevents attackers from creating huge recursive query trees that consume large amounts of CPU power and memory.

---

3. Securing against Alias/Batching Attacks

The Main Idea

In GraphQL, attackers can hide many operations inside a single HTTP request using aliases and batching. Traditional request counting becomes ineffective because the server sees only one request even though many actions are happening internally.

To solve this problem, the server stops measuring requests by quantity alone and starts measuring their internal complexity.

How the Protection Works

The server assigns a cost or weight to operations inside a query.

Simple actions are treated as low-cost operations, while expensive actions receive higher cost values. Operations that involve authentication, database lookups, or repeated processing are considered more expensive.

When a request arrives, the server calculates the total cost of everything inside the query.

Instead of asking:
“How many HTTP requests were sent?”

The server asks:
“How heavy is this request internally?”

Request Budget System

The server defines a maximum allowed budget for each request.

As the server analyzes the query, it adds together the cost of all included operations. If the total cost exceeds the allowed limit, the server rejects the request immediately.

This means attackers can no longer hide many expensive operations inside a single request.

Even though only one HTTP request is sent, the server recognizes that the request contains excessive internal activity and blocks it before execution.

The protection focuses on the actual workload created by the request rather than the visible number of requests.

Part 2: Securing REST API Vulnerabilities

REST APIs commonly depend on URLs, identifiers, and client-provided data. Many vulnerabilities happen because the server trusts user input too easily.

To secure REST APIs, the server becomes stricter about ownership verification, accepted input fields, and outgoing data exposure.

---

1. Securing against BOLA / IDOR

The Main Idea

A user being logged in does not automatically mean they should access every resource in the system.

The server must verify ownership of data instead of trusting resource IDs provided in the URL.

How the Protection Works

When a request arrives for a specific resource, the server does not simply search using the provided ID alone.

Instead, the server performs an additional ownership verification step.

The server checks two things together:

- Whether the requested resource exists
- Whether the currently logged-in user actually owns that resource

Both conditions must match before data is returned.

Why This Stops the Attack

If an attacker changes the resource number in the URL, the server will still verify whether that resource belongs to them.

Even if the resource exists in the database, the server refuses access if ownership does not match the logged-in session.

The attacker can no longer access random records simply by modifying numbers in the request path.

The protection removes trust from user-controlled identifiers and shifts trust toward authenticated ownership validation.

---

2. Securing against Mass Assignment

The Main Idea

Mass Assignment happens when the server automatically accepts and stores every field sent by the client.

To stop this, the server becomes selective about what information it accepts.

Instead of trusting all incoming data, the server creates a strict list of allowed fields.

How the Protection Works

When the client sends a JSON request, the server examines the incoming fields carefully.

The server does not forward the entire request directly to the database anymore.

Instead, it manually selects only approved fields that are safe to update.

For example, the server may allow changes only to profile-related fields while ignoring administrative or internal fields completely.

Why This Stops the Attack

If an attacker adds hidden fields into the request manually, the server simply ignores them because those fields are not part of the approved list.

The extra data never reaches the database.

The protection works by filtering input strictly and refusing to trust all incoming fields automatically.

The server accepts only what it explicitly expects and silently discards everything else.

---

3. Securing against Excessive Data Exposure

The Main Idea

The server should send only the exact information required by the frontend and nothing more.

Instead of returning full database records, the backend creates a safe public version of the data.

How the Protection Works

The backend defines a limited public representation of each object.

This public version includes only fields that are safe for users to view.

Sensitive information such as internal notes, passwords, hidden identifiers, or private records are excluded completely before the response is prepared.

When data is requested, the server first converts the internal database record into the safe public format and then sends that smaller version to the client.

Why This Stops the Attack

Even if attackers inspect the raw network traffic using browser developer tools, the hidden sensitive fields are not present in the response at all.

The server never sends unnecessary information to the client.

This follows a strict “Need to Know” principle where users receive only the specific data required for the application screen to function.

Sensitive backend information remains inside the server and never leaves its memory during the response process.